AUTOMAÇÃO DE TESTES WEB

Cypress: do cenário ao spec

Construa comandos, confira resultados e aprenda a investigar uma falha sem confundir cliques com qualidade.

O caso Fikily

Ao selecionar Acessórios, buscar Aurora e aplicar, a lista pronta deve exibir somente acessórios. A massa fictícia tem Mochila Aurora, Estojo Aurora e Teclado Aurora. O último é um eletrônico.

Etapa	O que você vai praticar
01 / Definir	Traduzir uma regra em resultado esperado.
02 / Construir	Estrutura, seletores, ações e assertions.
03 / Observar	Comparar uma referência e uma candidata.
04 / Revisar	Sincronização, isolamento e diagnóstico.

Antes de começar

É útil conhecer cenários e assertions do módulo Fundamentos. A apostila é opcional e não muda seu XP. O laboratório do QA Quest é uma simulação local: **não instala nem executa Cypress**.

Os exemplos usam dados fictícios e uma aplicação de teste autorizada. A execução real exige um projeto Cypress configurado fora do QA Quest.

01 / ESTRUTURA E ELEMENTOS

Dê intenção ao seu teste.

Um spec é um arquivo de teste. describe organiza um grupo de casos e it descreve um comportamento verificável. cy.visit abre a página do ambiente configurado. [1, 2]

```
describe("Catálogo Fikily", () => {
  it("filtra acessórios por busca", () => {
    cy.visit("/catalog")
    // Ações e assertions ficam aqui.
  })
})
```

Ambiente e dados controlados

/catalog é uma rota de exemplo. Ela só funciona se existir no servidor de uma aplicação de teste e se o projeto Cypress tiver baseUrl configurado. Não é uma rota pública prometida pelo QA Quest. Cada caso deve preparar seu próprio estado. [2, 3]

Consulta	Quando faz sentido
cy.get('[data-testid=…]')	Localizar por um atributo estável combinado com o time.
cy.contains('Aplicar')	Localizar pelo texto quando ele é parte relevante do contrato.
Classe visual ou posição	Evitar depender de cor ou ordem quando isso não faz parte da regra.

Os atributos de teste também podem mudar; precisam de manutenção. Papel e nome acessível são outra opção conforme a versão e a API disponíveis. Não confunda comandos nativos com comandos de plugins. Esta aula usa cy.get e cy.contains. [4, 5]

Pergunta prática

Se o botão mudar de roxo para azul e continuar aplicando o filtro, seu teste deveria falhar? Se não, uma classe de cor provavelmente não é o contrato certo.

02 / AÇÕES E RESULTADOS

Interagir não é comprovar.

As ações precisam de uma ordem coerente: o select deve existir, a busca precisa ser preenchida e Aplicar deve ser acionado antes de conferir os resultados.

```
cy.get('[data-testid="category-filter"]')
  .select("accessories")
cy.get('[data-testid="catalog-search"]')
  .type("Aurora")
cy.get('[data-testid="apply-filter"]').click()
```

Comando	Papel no exemplo
select	Escolher o value accessories em um select HTML. [6]
type	Digitar Aurora no campo da busca. [7]
click	Acionar Aplicar. O clique não comprova a lista resultante. [8]
should	Comparar um estado observado com o critério esperado. [9]

Assertion insuficiente

Conferir só que a página está visível ou que o primeiro produto é um acessório pode deixar um eletrônico passar despercebido. Confira uma lista não vazia e a categoria de todos os cards.

Depois de uma ação que pode atualizar a tela, consulte novamente os elementos necessários. Não suponha que uma referência antiga ao DOM continuará válida. [8]

Todos os nomes desta massa contêm Aurora. O caso evidencia o filtro de categoria, mas não cobre sozinho toda a busca, termos sem resultados ou todas as combinações de filtros.

03 / ASSERTIONS E SINCRONIZAÇÃO

Espere pelo estado, não por sorte.

O contrato marca a lista com data-state=ready após a atualização. O teste aguarda essa condição com limite e consulta os cards novamente. [9, 10]

```
describe("Catálogo Fikily", () => {
  it("filtra acessórios por busca", () => {
    cy.visit("/catalog")
    // Contrato: [data-testid="category-filter"]
    cy.get('[data-testid="category-filter"]').select("accessories")
    cy.get('[data-testid="catalog-search"]').type("Aurora")
    cy.get('[data-testid="apply-filter"]').click()
    cy.get('[data-testid="catalog-results"]', { timeout: 2000 })
      .should("have.attr", "data-state", "ready")
    cy.get('[data-testid="product-card"]').should(($cards) => {
      expect($cards.length).to.be.greaterThan(0)
      Array.from($cards).forEach((card) => {
        expect(card.getAttribute("data-category"))
          .to.equal("accessories")
      })
    })
  })
})
```

Como ler a retryability

Consultas encadeadas e assertions podem ser refeitas até passar ou atingir o timeout. Ações não são repetidas como assertions. O callback de should deve apenas conferir dados: não coloque cliques nem comandos Cypress dentro dele. Uma pausa fixa, como cy.wait(300), não garante que a atualização terminou. [9, 10]

Exemplo para um projeto configurado; não é executado no QA Quest. Um timeout limita a espera, não define um requisito de performance.

04 / DIAGNÓSTICO E MANUTENÇÃO

A mesma cor vermelha, causas diferentes.

Os perfis abaixo são fixtures locais independentes. Os tempos são dados didáticos, não medições reais do navegador.

Perfil	Esperado / observado	Leitura de QA
Referência + teste confiável	Esperado: lista pronta com acessórios. Observado: em 800 ms, dois acessórios.	O caso atende à regra, somente neste escopo.
Candidata + teste confiável	Esperado: nenhum eletrônico. Observado: em 800 ms, lista pronta ainda contém Teclado Aurora.	Defeito do produto. A pessoa recebe um resultado fora da categoria.
Referência + teste frágil	Esperado: conferir após a atualização. Observado: captura o estado anterior em 300 ms; a lista correta aparece em 800 ms.	Falha do teste: leitura prematura, não defeito do filtro.

O teste frágil desta aula

A variante com pausa fixa lê um snapshot em 300 ms e o confere sem retry. Ela não representa o spec confiável da página anterior. Em Cypress real, a cadeia e o tipo de assertion determinam o que será repetido. [10]

Revisão antes de aceitar o resultado

1. Reproduza com dados conhecidos; guarde esperado, observado e impacto.
2. Confira seletor, estado inicial, sincronização e alcance da assertion.
3. Não remova a assertion para esconder uma divergência.
4. Prepare cada teste de forma independente, sem depender de outro it. [3]

Parecer

A candidata precisa de ajuste no filtro e nova avaliação. Concluir a missão significa interpretar as evidências corretamente; não significa que uma suíte real foi executada ou que a aplicação está aprovada para produção.

05 / REVISÃO E PRÓXIMA PRÁTICA

Explique antes de automatizar mais.

Exercício: responda às quatro perguntas. Não é necessário instalar ferramentas.

1. Um clique passou. O que falta observar para validar o filtro?
2. Por que ler aos 300 ms pode gerar um falso alarme?
3. O que fazer se a lista pronta ainda exibir um eletrônico?
4. Por que um caso não deve depender do estado de outro?

Critérios de correção

1. Lista não vazia e categorias de todos os resultados.
2. A atualização pode não ter terminado; observar uma condição com limite.
3. Registrar evidência e devolver a candidata para investigação/ajuste.
4. Cada caso deve funcionar isoladamente, com estado inicial conhecido.

StepScript: prática complementar, opcional

Projeto de Gustavo Favero, criador do QA Quest. Grava interações e gera código Cypress/Playwright, mas não executa testes pelo painel. Use só ambientes autorizados e dados fictícios. Revise seletores, valores e assertions antes de compartilhar. A missão e o XP não dependem da extensão. [11]

Referências gratuitas e verificadas

[1] **Estrutura** - docs.cypress.io/app/core-concepts/writing-and-organizing-tests

[2] **cy.visit** - docs.cypress.io/api/commands/visit

[3] **Isolamento** - docs.cypress.io/app/core-concepts/test-isolation

[4] **Seletores** - docs.cypress.io/app/core-concepts/best-practices

[5] **cy.contains** - docs.cypress.io/api/commands/contains

[6] **select** - docs.cypress.io/api/commands/select

[7] **type** - docs.cypress.io/api/commands/type

[8] **click** - docs.cypress.io/api/commands/click

[9] **should** - docs.cypress.io/api/commands/should

[10] **Retry-ability** - docs.cypress.io/app/core-concepts/retry-ability

[11] **StepScript** - chromewebstore.google.com/detail/stepscript/kjapjhhghfigfilijhbdlchgpiojidhd

Conteúdo autoral didático, com paráfrases e exemplos próprios. Fontes consultadas em 01/11/2026. Confira a versão utilizada no seu projeto.